

Sprint 1 (Week 1): Core Setup and Donor Side

1. User Authentication and Verification → Signup and login system for donors and receivers, including phone number or NID based verification.
2. Post Food → Form for donor to create a food donation post.
3. Food Type, Quantity, Expiry Time, Location, Image Upload → Input fields attached to the post form.
4. Mobile Friendly → Base responsive layout so the app works properly on phones from day one.
5. Password Encryption → Store passwords using bcrypt or Argon2 hashing, never store plain text passwords. (1)
6. Basic Rate Limiting → Prevent bots from spamming signups or fake posts using request throttling.

Database for Sprint 1

1. Users table → id, name, phone, email, password_hash, role (donor/receiver/ngo/admin), verification_status, created_at
2. FoodPosts table → id, donor_id (foreign key to Users), food_type, quantity, expiry_time, location (latitude, longitude), image_url, status, created_at
3. Use PostgreSQL or MySQL for this relational structure, since food posts always belong to a specific user and have clear defined fields
4. Store images in cloud storage like AWS S3 or Firebase Storage, and only save the image_url in the database, never store raw image files inside the database

Sprint 2 (Week 2): Discovery and NGO Flow

1. NGO Role and Verification → Separate signup flow for NGOs, requiring registration documents, approved manually by Admin.
2. View Nearby Food, Search Food, Filter → Browsing and discovery features for available food posts.
3. Collection Request, Accept/Reject → NGO requests to collect food, donor approves or declines.
4. Status Update → Tracks food state through the donation lifecycle.
5. Map View → Displays pickup or NGO location visually.
6. CAPTCHA on Signup and Requests → Prevents bots from mass creating fake NGO or donor accounts.
7. HTTPS and API Token Authentication → All data between app and server travels encrypted, and every request must carry a valid token, so random bots cannot call your APIs directly.

Database for Sprint 2

1. NGOs table → id, user_id (foreign key to Users), organization_name, registration_document_url, verified_by_admin (boolean)

2. CollectionRequests table → id, food_post_id, ngo_id, status (pending/accepted/rejected), requested_at
3. Add a status column update in FoodPosts table with enum values like Available, Collected by NGO, Available at NGO Point, Taken, Expired
4. For location-based search, use a Geospatial Index, PostgreSQL has PostGIS extension for this, or MongoDB's geospatial queries if you choose NoSQL instead
5. Tokens table or use JWT (no separate table needed) for session and API authentication, JWT tokens do not require database storage since they are self-verifying

Sprint 3 (Week 3): Receiver Flow and Trust Features

1. Request Food, Anonymous Receiver, Pickup Code → Core receiver side interaction, including identity masking.
2. Notifications, Chat (Basic) → Communication layer between donor, NGO, and receiver.
3. Proof of Receipt Photo → Upload confirmation when food is collected.
4. Data Encryption for Identity Fields → Receiver's real identity, even though hidden publicly, should be stored encrypted in the database, not just hidden in the UI, so even a database leak does not expose real names.
5. Request Throttling per User → Prevent one account (bot or real) from spamming hundreds of food requests per minute.

Database for Sprint 3

1. FoodRequests table → id, food_post_id, receiver_id, is_anonymous (boolean), pickup_code, status, requested_at
2. Add encrypted_name and encrypted_nid columns in Users table specifically for receivers, encrypted using AES-256 before saving, and only decrypted when Admin needs to view them
3. Notifications table → id, user_id, message, is_read, created_at
4. Messages table (for Chat) → id, sender_id, receiver_id, food_post_id, message_text, sent_at
5. ProofOfReceipt table → id, food_request_id, image_url, uploaded_at
6. Consider Redis for rate limiting counters, since it is fast for temporary per-user request counting, instead of hitting your main database for every throttling check

Sprint 4 (Week 4): Ratings, Reporting, and Polish

1. Rating Users, Report User → Trust and accountability system.
2. History, Share Post, Top Donors Leaderboard → Engagement and transparency features.
3. Serving Log, Food Journey Tracking → NGO reporting and donor visibility features.
4. Final Security Audit → Check for exposed API keys, unencrypted fields, and missing authentication checks before launch.
5. Bot Detection on Leaderboard and Ratings → Prevent fake accounts from mass rating themselves up or gaming the Top Donors Leaderboard.

6. Bug fixes, UI polish, final testing

Database for Sprint 4

1. Ratings table → id, rater_id, rated_id, food_post_id, rating_value, comment, created_at
2. Reports table → id, reporter_id, reported_id, reason, status, created_at
3. ServingLogs table → id, ngo_id, food_post_id, people_served_count, served_at
4. FoodJourney can be derived from existing tables (FoodPosts, CollectionRequests, FoodRequests, ProofOfReceipt) joined together by food_post_id, no new table needed, just a combined query
5. For Top Donors Leaderboard, either run an aggregate query counting donations grouped by donor_id, or maintain a cached donor_stats table updated periodically for faster performance if user base grows